

Stage 1 – Project Introduction

Stateful Application-Layer Firewall & Intrusion Filter

Linux, Device Drivers, System Programming & C++

September 30, 2026

Abstract

This document outlines the foundational introduction, problem statement, scope, and expected outcomes for the Stateful Application-Layer Firewall & Intrusion Filter project. Designed as a high-performance system-level security daemon for Linux environments, the project addresses modern security vulnerabilities by combining low-level packet interception with deep stateful and application-layer (Layer 7) packet inspection.

1 Project Idea & Objective

The **Stateful Application-Layer Firewall & Intrusion Filter** is a robust, low-level network security framework engineered for modern Linux operating systems. Traditional network defenses often isolate kernel-level packet filtering from application-level semantic analysis. This project bridges that gap by building a modular, high-throughput security daemon and inspection engine entirely in modern C++.

Core Objectives:

1. Intercept live network traffic at the kernel-user boundary using efficient Linux networking subsystems (such as `Netfilter` and `libnetfilter_queue`).
2. Maintain rigorous, context-aware stateful connection tables to track TCP/UDP connection lifecycles and prevent spoofing or flood attacks.
3. Inspect application-layer payloads (e.g., HTTP headers, URIs, DNS queries) against customizable signature patterns and rule sets.
4. Provide a clean administrative interface and structured auditing subsystem for real-time security monitoring.

2 The Problem to be Solved

Traditional packet filtering tools (such as basic `iptables` rules or stateless firewalls) operate strictly at the Network and Transport layers (Layers 3 and 4 of the OSI model). They evaluate attributes such as source/destination IP addresses, port numbers, and basic TCP flags. However, the threat landscape has shifted dramatically:

- **Sophisticated Application-Layer Attacks:** Modern exploits—including SQL injection (SQLi), cross-site scripting (XSS) payloads embedded within HTTP headers, unauthorized API endpoint enumeration, and command injection—are encapsulated within legitimate-looking TCP streams on standard ports (such as ports 80 and 443). Layer 3/4 firewalls are completely blind to these payloads.

- **State Blindness and Session Vulnerabilities:** Stateless filters lack context regarding connection progression. Without tracking whether a packet belongs to an established, valid session, systems remain vulnerable to session hijacking, blind spoofing, and advanced persistent threats (APTs).
- **Performance vs. Depth Trade-offs:** Enterprise intrusion detection systems (IDS) often introduce significant latency or require massive user-space overhead, making them impractical for lightweight edge devices or high-throughput local servers.

There is a critical need for an optimized system-level architecture that combines rapid packet interception with a lightweight, highly extensible C++ application-layer inspection engine.

3 Project Scope

To ensure successful delivery and maintainable code quality, the project scope is explicitly divided into in-scope components and out-of-scope boundaries.

3.1 In-Scope Components

- **Packet Interception Subsystem:** Utilizing Linux networking primitives (`Netfilter` hooks via `libnetfilter_queue`) to intercept, examine, and issue verdicts (`NF_ACCEPT`, `NF_DROP`) on network packets in real time.
- **Stateful Connection Tracking (`conntrack`):** Implementing a custom connection table mapping flows using 5-tuples (Source IP, Destination IP, Source Port, Destination Port, Protocol) and managing state transitions (`NEW`, `ESTABLISHED`, `CLOSED`).
- **Application-Layer (L7) Parser:** Developing modular parsers for unencrypted plaintext protocols (focusing on HTTP metadata, request methods, and URI structures) to detect malicious signatures.
- **Configurable Rule Engine:** Building a dynamic rule-matching engine that supports pattern matching, regex signature evaluation, and threshold-based rate limiting without daemon restarts.
- **Management CLI & Logging:** Providing a command-line utility for administrative control and writing structured, timestamped audit logs to disk.

3.2 Out-of-Scope Boundaries

- Full live decryption of TLS/SSL encrypted streams without a dedicated man-in-the-middle proxy (inspection is restricted to unencrypted payloads, handshake metadata, and Server Name Indication analysis).
- Large-scale commercial signature databases (focus is placed on architectural soundness and custom rule sets rather than thousands of commercial Snort rules).

4 Expected Outcome & Target Applications

4.1 Expected Deliverables

1. A fully operational, multithreaded C++ system security daemon executing under Linux.
2. A modular codebase with strict separation of concerns across packet capture, state tracking, and signature inspection.
3. Comprehensive technical documentation, UML diagrams, and testing benchmarks verifying throughput, low latency, and reliability.

4.2 Target Applications

- **Edge Security Gateways:** Lightweight perimeter defense for custom Linux-based routers and IoT environments.
- **Host-Based Intrusion Detection (HIDS):** Protecting critical backend servers and microservices against application-layer abuse.
- **Academic Research:** Demonstrating advanced system programming, C++ memory management, and Linux kernel-user space communication.

5 Conclusion & Next Steps

Stage 1 establishes the conceptual foundation and architectural boundaries of the Stateful Application-Layer Firewall. With the project vision and scope locked in, we are ready to ad-

vance to **Stage 2 – Project Requirements & Development Plan**, where we will formalize functional requirements, non-functional constraints, and the development timeline.